MNXB11: Project Report

David Chelidze, Estrella González Gómez, Joel Johansen,
Maja Solman Wålsten and Evelina Vedin Meijer

October 2026

1 Introduction

The project aims to create a repository that allows the user to build a program that performs one of four analyses of temperature data collected from a SMHI (Sveriges meteorologiska och hydrologiska institut) station located in Lund. To accomplish this, the group agreed to assign one member as the release manager and all other members then sent pull requests to the release manager's remote upstream. A detailed description of the analyses can be found below.

2 Data extraction

Before any data could be analyzed, it first had to be parsed (reformatted so that it is readable by a program) and extracted. This first part was automated using a BASH script, whose data parsing part is

```
# Parse data using awk with delimiter ;
awk -F';' 'NR > $skip_rows {

    # Only include data that is controlled
    if ($4 == "G") {

        # Split date into year, month and day and store in
        # an array called date
        split($1, date, "-")

        # Split time into hour, minute and second and store in
        # an array called time. Only the hour will be used
        split($2, time, ":")

        # Print formated data to stdout
        printf("%i,%i,%i,%i,%0.1f\n", date[1], date[2], date[3], time[1], $3)

    }
}' $raw_data_path >> $parsed_data_path
```

The *awk* $-F'delimiter'$ $'\{script\}'$ *input_file* >> *output_file* command takes as input a file with raw data, whose path is stored in $\$raw_data_path$, and reads all rows with index larger than $\$skip_rows$. The rows are separated using the delimiter ';' so that the four columns of the raw data file gets stored in four variables that is accessed by the *awk* script. The if statement filters out all data that is not "good". Any data that is "good", on the other hand, gets parsed using the $split(input, array, delimiter)$ function, which splits the string *input* into parts delimited by *delmiter* and stores the result in an array named *array*. The formatted data then gets printed to stdout and appended to the parsed data file, with path stored in $\$parsed_data_path$.

The parsed data resulting from the above was then additionally processed into a ROOT TTree with a single brach and five leafs corresponding to the year, month, day, hour and temperature of the observation. In doing so, a custom class called *Observation* derived from the TObject (necessary to use non-class types such as Int_t and $Double_t$) class was made

```
class Observation : public TObject {

    public:

        // Default constructor with zero parameters
        // This is required to use I/O objects
        Observation();

        // Constructor with parameters
```

```
        Observation(Int_t year, Int_t month, Int_t day,
                Int_t hour, Double_t temperature);

        // Override destructor from base class
        ~Observation() override;

        // Getters
        Int_t Year() const;
        Int_t Month() const;
        Int_t Day() const;
        Int_t Hour() const;
        Double_t Temperature() const;

        // Set function
        void SetObservation(Int_t year, Int_t month, Int_t day,
                Int_t hour, Double_t temperature);

    // Member variables
    private:

        Int_t _year;
        Int_t _month;
        Int_t _day;
        Int_t _hour;
        Double_t _temperature;

        // Observation class
        ClassDef(Observation, 1);

};
```

The class has five private variables that will be used to write the leafs to the ROOT TTree. Getters, setter, destructor and constructors were defined in a separate source file. Since the class do not make use of any dynamically located memory the destructor definition was left empty.

In addition to the above class, another translation unit was written that used the above custom class in order to create a ROOT dictionary which then itself was used to create the desired ROOT TTree. The header file of this translation unit features two functions

```
std::vector<Observation> parse_observation(const std::string& csv_file);

void write_observation(const std::vector<Observation> &observations,
        const std::string& tree_file);
```

The first function *parse_observation* takes as input the parsed data file and returns a vector of *Observation* objects corresponding to the data in the parsed data file. This function makes use of an external csv parser library. The second function *write_observation* takes as input a vector of *Observation* objects as well as a file name to which the ROOT TTree will be written. Similarly, the definitions of these were written in a separate source file.

3 Generating a ROOT dictionary and compiling library using make

Before building the main program one first have to generate the ROOT dictionary and compile the source file created in doing so. To automate this process, rootcling (a tool to generate ROOT dictionaries) was embedded into the Makefile. The relevant parts in the Makefile looks like

```
HEADERS := include/Observation.h
SOURCES := src/Observation.cxx
ROOTSYS := $(shell root-config --prefix)

ObservationDict.cxx: $(HEADERS) include/Linkdef.h
    rootcling -f $@ -c -Iinclude -Iexternal/include
    -I$(shell root-config --incdir) -p $^
```

```
libObservation.so: ObservationDict.cxx $(SOURCES)
    $(CXX) -fPIC -shared -o$@ $(CXXFLAGS) -I$(ROOTSYS)/include
    $^ $(shell root-config --ldflags --libs)
```

When you execute *make libObservation.so* from the base directory, rootcling will first generate the dictionary source file *ObservationDict.cxx* for the header *Observation.h*. The *Linkdef.h* file contains information about which classes should be added to the dictionary. In this case we only add the *Observation* class. Then this dictionary gets compiled together with *Observation.cxx* into the library *libObservation.so*. When building the main executable you then also hove to include the *ObservationDict.cxx* as a source file. The *libObservation.so* was also added to the *rootlogon.C* file so that it automatically loads whenever a ROOT session is initiated.

4 Analysis 1: Warm summer days in Lund

The aim of this analysis was to study how the number of warm summer days in Lund varies between years. We used the period June–August and defined a warm day as a day where the highest available temperature measurement reached at least 25 °C.

Since several measurements can exist for the same day, each calendar day was counted only once. Years with fewer than 92 available summer days were excluded to avoid treating missing data as cold days.

The result shows a large variation between individual summers, with some years having only a few warm days and others having more than 20. No simple increasing trend is clearly visible from the scatter plot alone. A limitation is that the observation times changed during the historical period, so the highest available measurement is not always the true daily maximum.

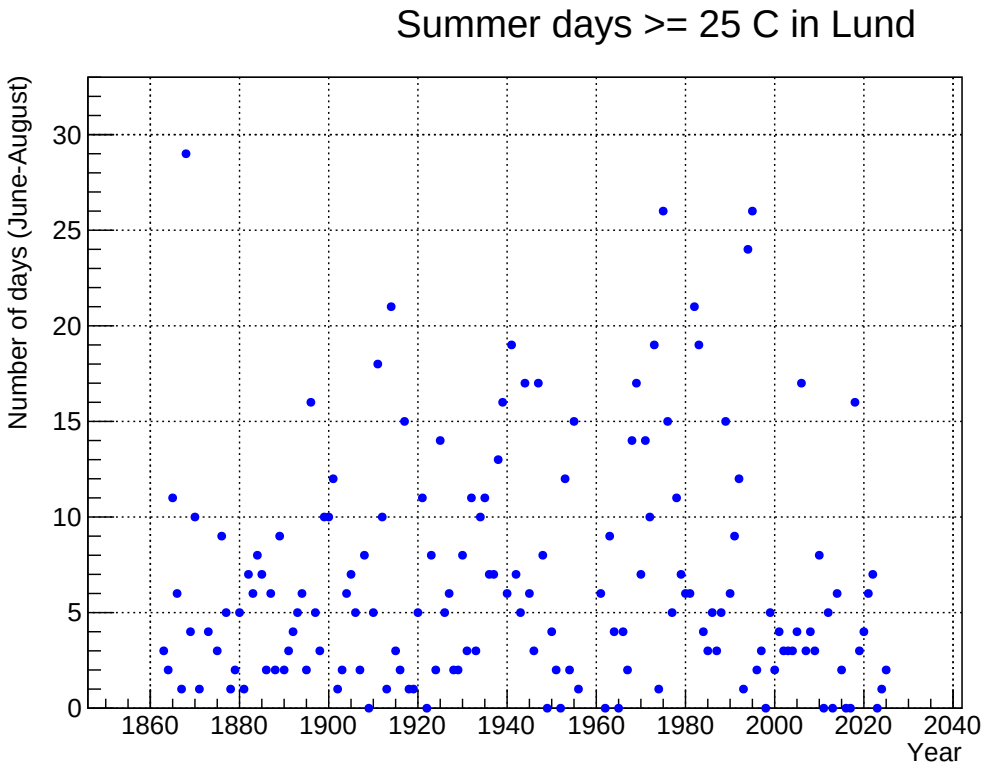


Figure 1: Number of summer days in Lund where the highest available temperature measurement reached at least 25 °C.

5 Analysis 2: Daily temperature throughout the year

The aim of this analysis was to investigate how the mean temperature in Lund varies throughout the calendar year and how much the temperature varies between different years for each day.

5.1 Method

Temperature measurements were grouped by calendar date. For each date, the available measurements were used to calculate the daily mean temperature. These daily means were then grouped by day of the year, allowing the mean temperature and standard deviation to be calculated for each calendar day across the available years.

February 29 was excluded to obtain a consistent 365-day calendar. This avoids the need to accommodate an additional day in leap years when comparing the same calendar day across different years. The analysis produced a mean temperature and a standard deviation for each day of the year.

5.2 Results

Figure 2 shows the mean temperature throughout the year. The red line represents the mean temperature for each calendar day, while the shaded band represents one standard deviation above and below the mean. The band illustrates the variability in daily mean temperatures between years: a wider band indicates greater variability, whereas a narrower band indicates that the temperatures are more consistent.

The mean temperature follows a clear seasonal pattern, increasing during spring and reaching a maximum of approximately 18 °C around days 200–210 of the year, corresponding to late July. Temperatures then decrease throughout autumn and approach 0 °C towards the end of the year. The shaded band indicates that variability between years changes throughout the calendar year, with differences in its width reflecting differences in the standard deviation of daily mean temperatures.

5.3 Limitations

The analysis is based on the available observations, and the number of observations may differ between calendar days. Consequently, the precision of the estimated means and standard deviations may also vary throughout the year. Furthermore, the standard deviation describes the variability between years and should not be interpreted as the uncertainty of the estimated mean.

In addition, the calculated daily means depend on the available measurement times and may therefore differ from official daily mean temperatures. Differences in data coverage between years may also affect the estimated variability.

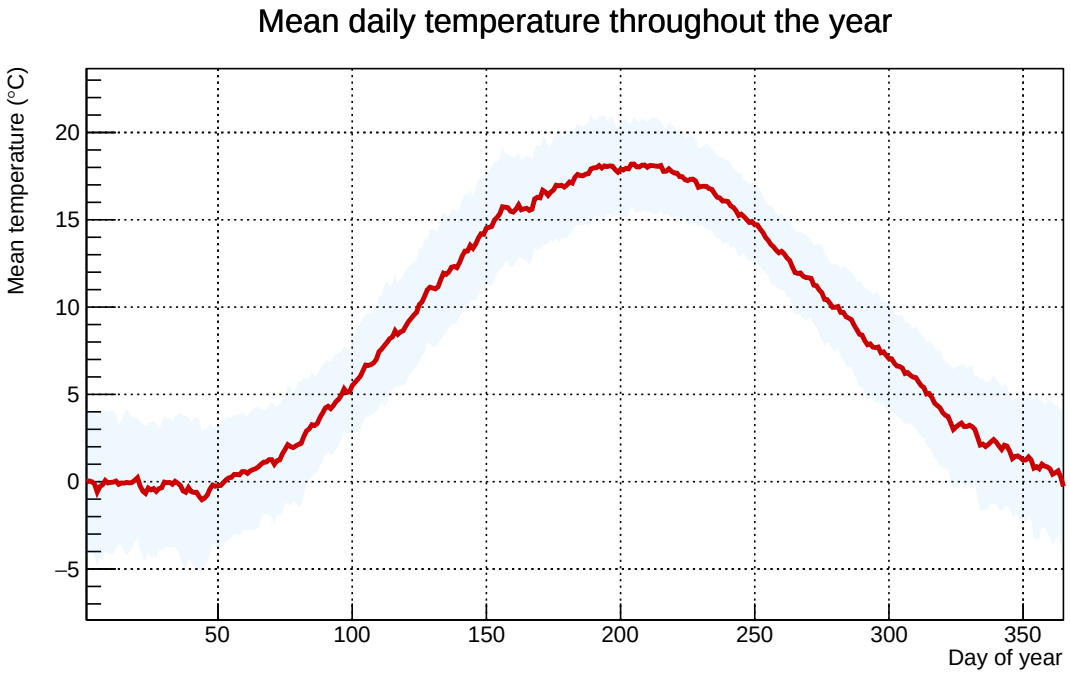


Figure 2: Mean daily temperature throughout the calendar year. The red line represents the mean temperature for each calendar day, and the shaded band represents one standard deviation above and below the mean.

6 Analysis 3: Annual mean temperature in Lund

The aim of this analysis is to use data from SMHI to predict the mean temperature in Lund of future years and analyze if this can be used to draw any conclusions about global warming.

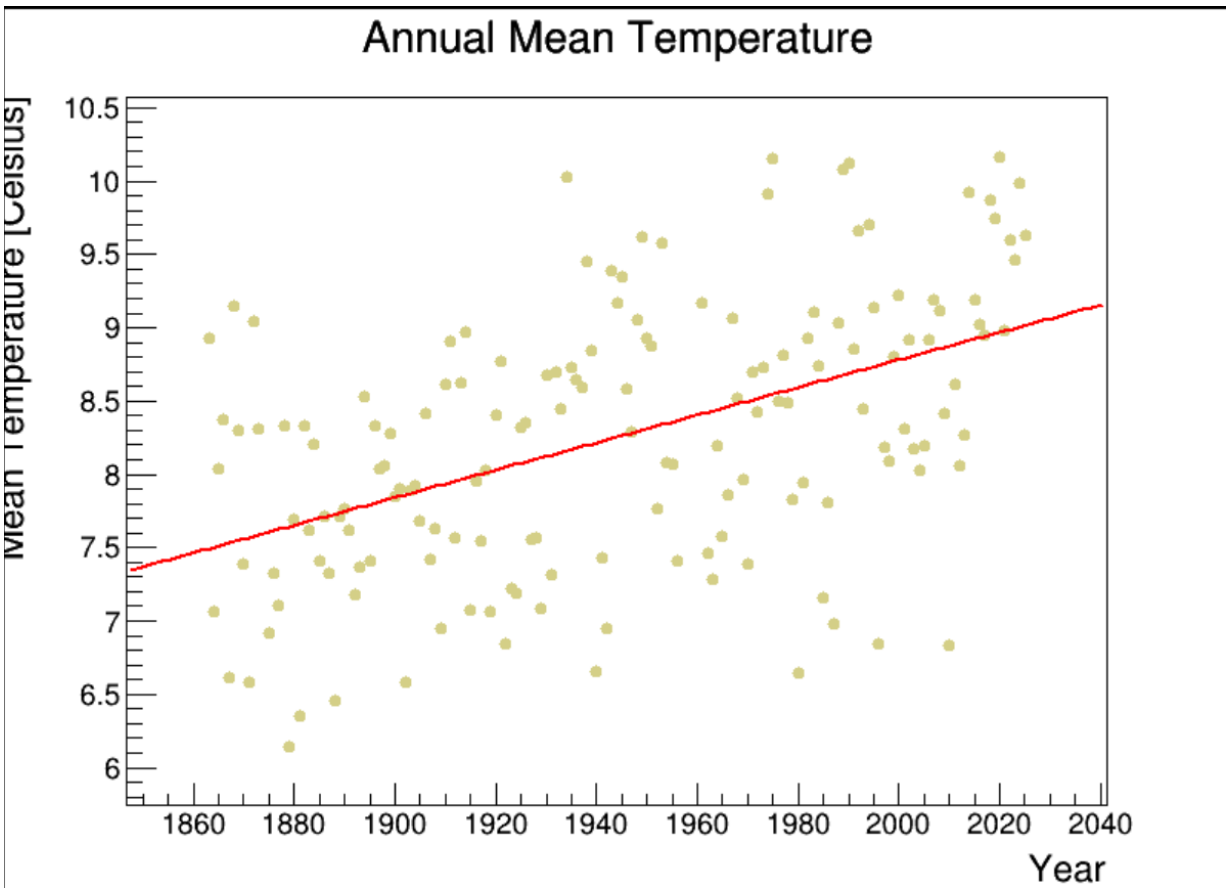


Figure 3: The figure illustrates the annual mean temperature as datapoints. A line which is fitted to the datapoints is marked in red. This is used to predict future annual temperatures, as a function of the current trend of temperature increase.

6.1 Method

The mean temperature of each year is computed using the sum of all good? temperature measurements for each year, divided by the total number of measurements that same year. This calculated mean is then used to create a plot with all datapoints, see Figure 3. As it can be seen in the figure, the datapoints suggests a more or less steadily increasing mean temperature. Therefore a line is fitted to the datapoints and is then used to predict the mean temperature of a given year in the future, if the yearly temperature continues to follow the current trend. blablabbla.... suggesting an increasing temperature.

6.2 errors and limitations

The code has limitations so that only years with at least 360 days of data is included. This is done so that years with e.g. only measurements from the winter months is not included, as this would result in a datapoint suggesting an annual temperature far below average. The need for such a limitation was discovered when the generated plots contained datapoints for mean annual temperatures way below average. Looking at the data it was then discovered that those years only had measurements taken during the winter months, making limitations of years included in the analysis necessary.

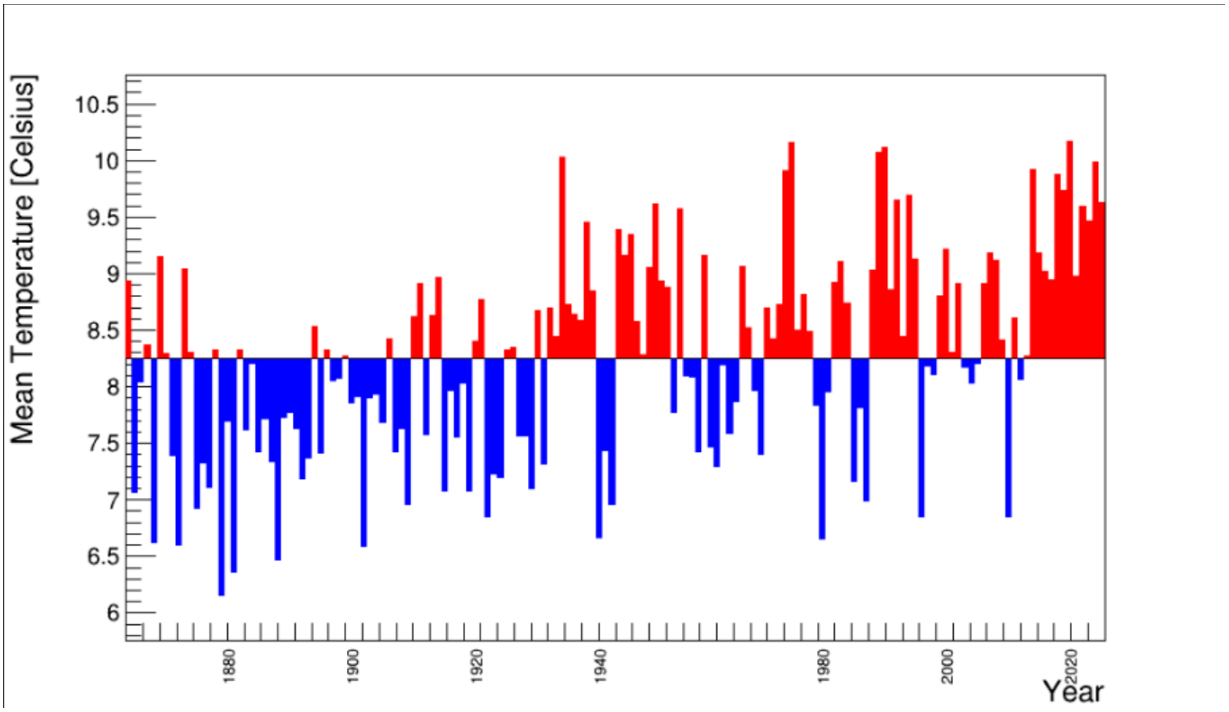


Figure 4: The figure illustrates the annual mean temperature as deviations from the total average.

6.3 Results

Year	Temperature [° C]
2030	9.06
2035	9.11
2040	9.16
2045	9.20
2050	9.25
2055	9.30
2060	9.35

Table 1: A table of predicted future annual mean temperatures

As can be seen in Figure 3 the fitted line suggests a trend of increasing temperature. Though the increase in temperature is of course not linear, it can still be concluded that recent years have a higher frequency of warmer years as well as much fewer years with a mean temperature below average. Although, even if the current trends might suggest a rather linear increase in temperature, this might not hold in a few years time. Especially considering we are in Sweden. If we were to do the same analysis of the mean temperature but with data from all over the Earth, this prediction of increasing temperature might hold. But locally, a global warming does not inherently mean a local increase in temperature. Sweden might even get colder as climate feedback loops could slow down the gulf stream, resulting in much colder winters in northern Europe. Therefore, to actually being able to draw predictions about future temperatures locally, much more would be needed to take into consideration.

6.4 AI usage

In this analysis AI was used to discuss ????

7 Analysis 4: Warmest and coldest days

Describe aim of the analysis, method, results and so on. The aim of analysis 4 is to investigate when during the year the warmest and coldest days typically occur in Lund. The general method is to identify, for each year which calendar day was the annual warmest and coldest, respectively and then count how frequently each calender day was the warmest or coldest. These results are visualized in two overlapping histograms, a red for

the warmest and blue for the coldest. In the plot, the x-axis represent the days of the year (1-365), and the y-axis represents the number of years which that day was the warmest or coldest.

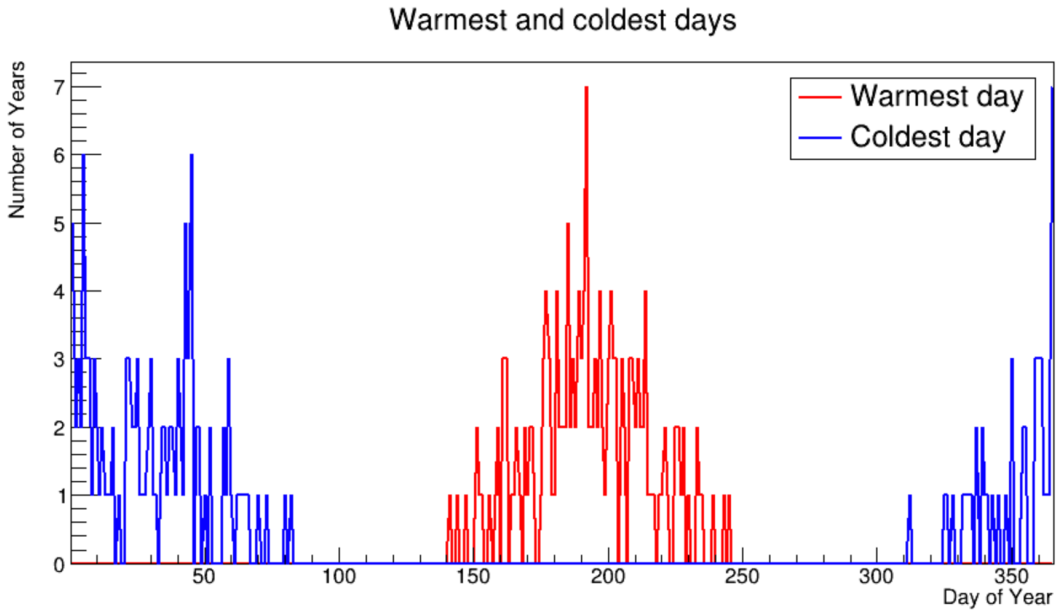


Figure 5:

7.1 Method

In order to retrieve these results the data has to be processed, firstly by reading the temperature measurements from the ROOT TTree, then grouped together by year and day of year. The calender days throughout each year is assigned their respective number from 1-365. In this process, February 29 is excluded in order to make dates comparable between regular years and leap years. When the daily data is sorted, daily averages are calculated since the number of measured temperatures varies from 0-3 measurements per day. Years that have less than 360 days of data are excluded in order to reduce the influence of incomplete years. Then for each year that has enough measurements, the day with the highest and lowest average temperature is found. The histogram is then created with 365 bins which are filled with the counts. For this analysis, 158 years between lala and lala satisfy the requirements and is shown in the histogram.

7.2 Results

From the histogram 5 it can be concluded that the warmest days occurs some day between (date) to (date) and and that the warmest days occurs most frequently in (month). The histogram for the warmest days can be compared with a normal distribution with a peak at (date). The coldest day on the other hand occurs some day in between (date) and (date), also the coldest days occur most frequently in (monthS?). When comparing the two histograms it is clear that the plot for cold days does not have the normal distribution like the plot for the warm days. Instead it has an even distribution but with two distinct peaks one around (date) and one around (date). (Discuss no overlaps in histograms.) (Explain the results, seasonal temp variations, osv)

7.3 Limitations

Discuss limitations and possible improvements, years with fewer than 360 (why 360?) recorded days are excluded, feb 29 (no worries), 1-3 measurements per day at different times, many years into one distribution-¿compare different historical periods to each other.